

ML model serving platform: versions + A/B, global clients

front: what to ask, what to design · HLD complete by minute 15 · one page

1 REQUIREMENTS 0–5 min · questions first, then sentences

- **Ask:** who calls (apps, services)? one call = request/response or stream? how many models, what sizes (MB to 100 GB)? latency per model class?
- **Ask:** peak req/s, per region? how many regions, may a call cross regions? A/B = % split, sticky by user or customer, canary %? who owns the GPUs, is cost a constraint? what is "rollout done"?
- **An engineer can:** register a model · upload a version · deploy it to a pool/region · run an experiment between two versions · roll out / roll back. **A client can:** call one stable endpoint per model and get a result or stream from the version the rules choose. **Operators** see per-version, per-arm health.
- **Technical challenges:** sizes differ 1000× · loading 100 GB takes minutes, never on the request path · one registry, global clients: rules may be seconds stale, calls never fail on it · sticky A/B as a cheap hash · GPU cost = batching + bin-packing · availability on inference, consistency in the control plane.
- **Anchors:** 10k req/s peak · 100 models, 100 MB–100 GB · p99 100 ms small models · 3 regions · 99.99%.

2 CORE ENTITIES 5–9 min · list, then fields

- **Model**(id, name, owner, aliases prod → v7, canary → v8)
- **ModelVersion**(model, version, artifact URI, checksum, size, GPU memory, status uploaded|validated|deployable|retired) — immutable
- **Deployment**(version, region, pool, replicas, GPU class, generation, status)
- **Experiment**(model, arms[], key user|customer, stop criteria) · **Arm**(experiment, version, weight %, pinned customers[])
- **Replica**(node, GPUs, version, state pulling → loaded → warm → ready(v, G) → serving → draining)
- Not entities: the request (metered), the batch (runtime), the region (partition key).

3 API 9–13 min · one endpoint per requirement

- **POST /models** · **POST /models/{id}/versions** (signed upload URL, then validation)
- **POST /deployments** {version, region, replicas, gpu_class, canary_pct} · **POST /experiments** {model, arms, key} · **/rollouts/{id}/pause | resume | rollback**
- **POST /v1/models/{name}:predict** {inputs, version?: prod | v8, user_key?} → result or stream; response carries **served_version + arm**
- **GET /models/{id}/versions/{v}/status** → loaded · ready · serving per region
- internal: **POST /replicas/{id}/report** {generation, version, state} · routers watch a rules snapshot per model

4 DATA FLOW 13–20 min · deploy path, then one request

- **Deploy:** register v8 → weights + checksum in the artifact store → controller places (bin-pack by GPU memory, topology-aware), generation G → agent pulls chunks, verifies, NVMe, GPU memory, warm-up → ready(v8, G) → ready set + rules (canary 10% → v8) → routers pick up in seconds.
- **Request:** client → nearest region (anycast) → gateway (auth, quota, meter) → router: alias → version → arm (sticky hash of key) → ready replica here → per-version queue → batcher flush (size or time) → GPU → result/stream back, stamped **served_version + arm** → metrics + experiment log async.
- **No ready replica in this region:** fail over to the next region, never cold-load on the request path.

5 HIGH-LEVEL DESIGN 20–33 min · one board, planes meet at the router

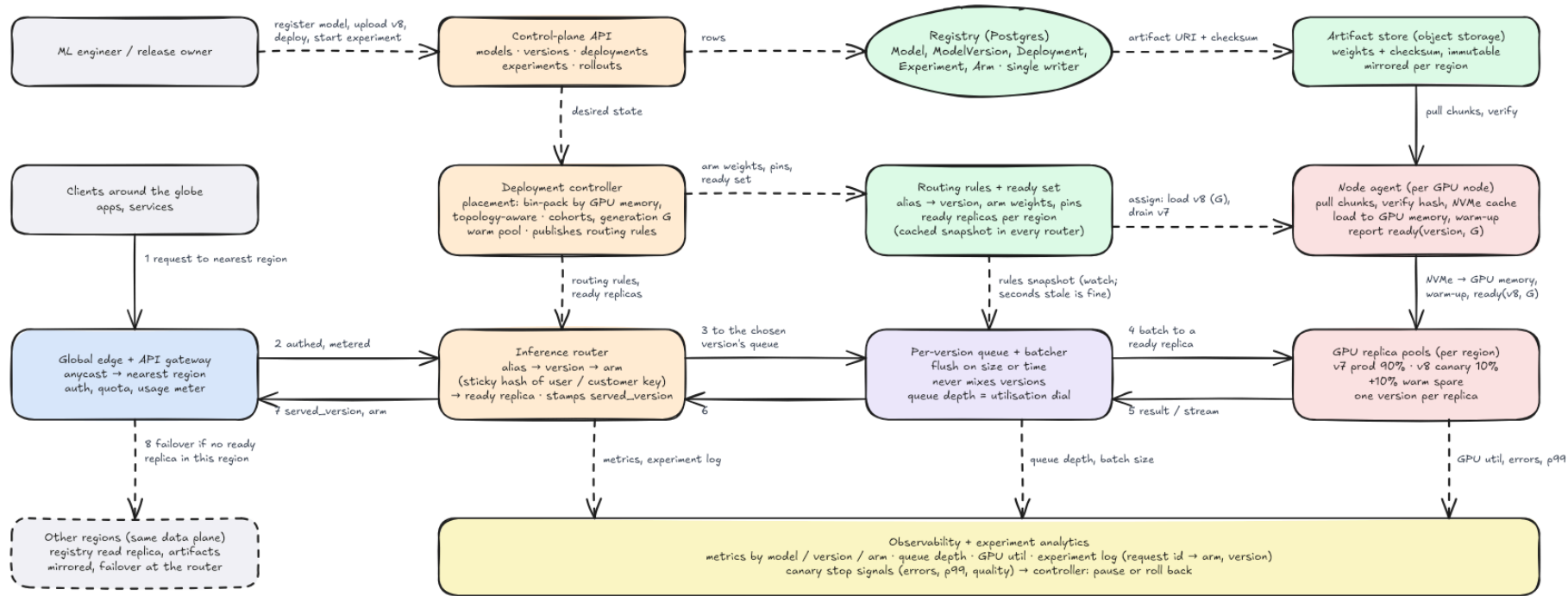
- Row 1 control: engineer → API → registry (Postgres, single writer) · artifact store (immutable, mirrored).
- Row 2 deployment: controller (placement, cohorts, generation, warm pool) → rules + ready set → node agent (pull, verify, NVMe → GPU, warm-up, ready).
- Row 3 data: client → edge + gateway → router → per-version queue + batcher → GPU pools (v7 90%, v8 10%, +10% warm). Numbered path there and back.
- Row 4: observability + experiment analytics; canary stop signals → controller. Dashed "other regions" box.
- **Sum:** small model batch 32 / 10 ms → 3,200 req/s per GPU → 4 GPUs for 10k, run 6. 100 GB pull at 10 Gbit/s = 80 s + NVMe → GPU ≈ 20 s + warm-up → minutes: a version switch is control-plane only.

6 DEEP DIVES 33–45 min · offer, then follow the interviewer

- **GPU optimisation, on the board:** deploy = bin-pack by memory + headroom, topology-aware, FP8/INT8, tiered GPU classes; run = dynamic batching with a latency budget, queue depth as the dial (70%), autoscale on depth + util, scale-to-warm. One line: latency vs utilisation, queueing is the lever.
- **Life of a request:** numbered path and the path back; batcher never mixes versions; retries idempotent.
- **Multi-tenancy:** one version per replica, per-model queues, per-customer quotas, shed by tier, no GPU memory oversubscription.
- **A/B:** sticky hash(key, experiment id); weights change only via the control plane; analysis reads the experiment log; per-arm stop signals → pause / rollback = new generation.
- **Multi-region:** single-writer registry, cached rules (registry down = no changes, inference continues); artifacts mirrored first; router fails over.
- **Bow:** control plane says which versions and how to split; deployment plane makes replicas exactly-version-ready; data plane routes, batches, streams back. Nothing on the request path waits on the registry or loads weights.

The board at minute fifteen, filled in by minute forty

back: the answer, as it should look on the shared screen



Failure / cold start: a large version is never loaded on the request path. Warm pool + NVMe pre-placement before the rollout; if a region has no ready replica, fail over (extra latency) instead of erroring.

Numbers to say: 10k req/s peak · 100 models, 100 MB–100 GB · small model batch 32 / 10 ms → 3,200 req/s per GPU → 4 GPUs for 10k; run 6 100 GB pull at 10 Gbit/s = 80 s + NVMe→GPU ≈ 20 s + warm-up → minutes, so a version switch is control-plane only queue target 70% util · rules stale by seconds is fine, calls never fail on the registry

Rollout: canary 10% → watch per-arm errors, p99, quality → 50% → 100% rollback = new generation, rules revert, v8 drains. A/B: sticky hash(key, experiment id); analysis reads the experiment log, not the router.

Legend

- client / other region
- edge, gateway
- service
- store / rules
- queue + batcher
- GPU nodes
- observability

solid = request bytes and weights dashed = control, reports, metrics